

Manual de Estudio: Sistemas de Gestión Empresarial (SGE)

Ecosistema Odoo 18, Programación en Python y Bases de Datos PostgreSQL

Implantación y Adaptación de ERP/CRM • Convocatoria: Junio 2026

BLOQUE 1: IMPLANTACIÓN, PARAMETRIZACIÓN Y METODOLOGÍAS DE GESTIÓN

En el despliegue corporativo de un sistema integrado, es de **vital importancia conocer en profundidad la actividad y los procesos** específicos de una empresa antes de comenzar la fase de implantación. Este conocimiento exhaustivo asegura que el software de gestión se configure y despliegue de forma correcta respondiendo a las necesidades reales de la organización. Esta necesidad metodológica se extiende a todo tipo de estructuras organizativas, incluyendo aquellas que carecen de fines lucrativos como las **ONG o empresas sociales**, las cuales requieren de igual modo un software de gestión robusto para controlar sus flujos y procesos económicos.

Durante el ciclo de vida del software, se debe diferenciar conceptualmente entre dos metodologías de adecuación funcional:

- **Parametrización:** Consiste en el proceso básico de configuración de ajustes preexistentes en el ERP para que el sistema se adecue a la empresa **sin necesidad de programar código profundo** (por ejemplo, la configuración del tipo de IVA aplicable a los productos). Es inusual que un ERP-CRM moderno no admita parametrizaciones, ya que todas las empresas presentan pequeños detalles específicos que deben recogerse como datos estructurados.
- **Adaptación:** Se refiere a un tipo de proceso que introduce un cambio mucho más profundo en la estructura del software, por lo que **requiere de forma normalizada de recursos específicos como desarrolladores de software** (por ejemplo, registrar el nombre de un entrenador de cada cliente en una empresa de gimnasios requiere adaptar el modelo incorporando un campo específico).

Para la gestión y planificación documentada del proyecto bajo metodologías ágiles, la arquitectura **SCRUM** emplea como documentación clave el **Product Backlog**. Este componente técnico funciona como una *****"lista viva" de requisitos funcionales y no funcionales priorizados**** que documenta de forma explícita todo lo que el cliente valora. Asimismo, la planificación se fragmenta en **Sprints**, descritos como ciclos cortos de desarrollo (generalmente semanas) orientados a validar partes funcionales del sistema de forma iterativa ante los usuarios implicados.

BLOQUE 2: ARQUITECTURA DEL FRAMEWORK ODOO 18

El software empresarial Odoo se cimenta sobre una arquitectura clásica **Modelo-Vista-Controlador (MVC)**. Las tablas lógicas de la base de datos subyacente se denominan técnicamente **Modelos** (representando la capa "M" del patrón).

El framework integra una potente capa de **Mapeo Objeto-Relacional (ORM)** que permite a los desarrolladores crear modelos de negocio complejos y gestionar de forma nativa la persistencia contra el motor de base de datos relacional **sin necesidad de escribir apenas sentencias SQL manuales**, aislando de forma limpia la lógica de negocio de la capa de presentación. Los objetos de negocio se hacen persistentes en el sistema extendiendo de forma obligatoria la clase base del framework: `models.Model`.

A nivel de sistema de archivos, la implementación y despliegue de extensiones propias se realiza dentro de la carpeta estándar de producción denominada `addons`. Todo módulo o componente personalizado que modifique o personalice el comportamiento estándar de Odoo requiere contar con dos ficheros principales obligatorios en su directorio raíz:

1. `__manifest__.py` (El Manifest): Archivo encargado de definir la totalidad de los metadatos de documentación del componente, como su nombre técnico, versión, dependencias con otros módulos y la lista ordenada de ficheros XML que se deben cargar en la base de datos.
2. `__init__.py` (Fichero de Importaciones): Fichero que gestiona la carga inicial inicializando las importaciones de paquetes, clases o submódulos internos de Python.

La creación física de la estructura base y el andamiaje de carpetas de un nuevo módulo se automatiza de forma estricta ejecutando el comando nativo desde el terminal de comandos:

```
odoo-bin scaffold mimodulo addons
```

Para que el sistema cargue de manera permanente los datos de demostración o registros de configuración declarados en los archivos de un nuevo módulo, el técnico debe proceder a **instalar o actualizar el módulo** dentro de la interfaz del sistema de gestión.

BLOQUE 3: DESARROLLO DE LA LÓGICA DE NEGOCIO EN PYTHON

Odoo utiliza el lenguaje **Python** para la codificación y desarrollo de la lógica computacional interna del sistema. Sintácticamente, se describe como un **lenguaje de alto nivel, interpretado y orientado a objetos**, caracterizado por poseer una sintaxis muy sencilla que facilita el aprendizaje y elimina detalles de bajo nivel como la gestión directa de la memoria. Además, cuenta con la propiedad de la **portabilidad**, garantizando su ejecución directa en múltiples plataformas tecnológicas de servidor (Linux, Windows o macOS).

Técnicamente, una de sus grandes ventajas en entornos orientados a objetos es su **ligadura dinámica**, lo que dota al desarrollo de una alta flexibilidad de comportamiento en tiempo de ejecución. Asimismo, soporta mecanismos avanzados de **herencia múltiple**.

3.1 Estructuras de Datos y Palabras Clave de Control

Los tipos de datos en Python se clasifican rigurosamente según la viabilidad de modificar su contenido en memoria tras su creación:

- **Mutable:** Su contenido interno se puede alterar directamente durante la ejecución. La estructura mutable por excelencia delimitada por **corchetes []** son las **Listas (list)**.
- **Inmutable:** Su contenido no puede alterarse directamente. Intentar modificar un tipo de dato inmutable (como las **Tuplas** o un valor numérico real de tipo `float`) provocará una excepción, obligando al intérprete a reasignar una nueva dirección de memoria si se varía el valor.

Para el manejo de colecciones dinámicas y optimización de datos, la estructura más adecuada para **evitar elementos duplicados** es el **Conjunto (set)**, caracterizado por ser una colección no ordenada de elementos únicos. Si se requiere que este conjunto sea estrictamente inmutable, se debe invocar formalmente la sentencia `frozenset(secuencia)`.

El control de flujo y la lógica condicional se operan mediante estructuras estandarizadas como `if-else`, `for` y `while`. Las operaciones booleanas complejas se manipulan mediante los operadores lógicos nativos `not` (utilizado para negar una condición booleana), `and` y `or`. El símbolo `#` se reserva para introducir comentarios aclaratorios dentro del código fuente. Cuando se requiere acceder a funciones matemáticas avanzadas fuera del núcleo básico (como funciones trigonométricas), se debe realizar la importación de la biblioteca estándar mediante `import math`.

3.2 Funciones y Parámetros del Modelo

Las funciones en Python se definen explícitamente mediante la sentencia clave `def`. El flujo del lenguaje diferencia taxativamente entre dos conceptos:

- **Función:** Utiliza de forma obligatoria la sentencia clave `return` para devolver un resultado estructurado al flujo principal del programa.
- **Procedimiento:** No especifica ninguna instrucción de retorno, por lo que técnicamente **devuelve automáticamente la constante None** (tipo de dato reservado en Python para representar un valor nulo).

Al declarar los atributos de los campos en las clases Python de Odoo, se exponen modificadores funcionales: el atributo `required=True` define de forma estricta que el campo no puede estar vacío en la base de datos al validar el registro; el atributo `index=True` fuerza al motor a crear un índice indexado para optimizar el rendimiento de las consultas; y el atributo `help="..."` inyecta una descripción informativa contextualizada para guiar al usuario final en la interfaz gráfica.

BLOQUE 4: DISEÑO DE INTERFACES (VISTAS Y QWEB)

En la arquitectura del frontend de Odoo, las interfaces de usuario se sustentan sobre una separación clara de responsabilidades operativas: las **Vistas** se configuran como listados interactivos o cuadros lógicos para visualizar datos almacenados directamente en la pantalla del sistema, mientras que los **Informes (report)** se definen como documentos estáticos destinados estrictamente a ser imprimibles o descargables.

Las vistas se definen estructuralmente mediante código **XML** y son interpretadas de forma transparente en el cliente por el framework de JavaScript. En las versiones modernas de Odoo, el motor visual utiliza de manera prioritaria el framework de componentes de código abierto **OWL (Odoo Web Library)** para la construcción de interfaces de usuario altamente reactivas y modernas en la web.

4.1 Estructura XML de Formularios y Listados

Las interfaces de captura de datos interactúan mediante el uso técnico de etiquetas XML raíz bien definidas:

- La etiqueta `<form>` se utiliza como raíz mandatoria para estructurar los formularios de datos, los cuales representan el **mecanismo principal de interacción** para introducir información, dar de alta registros, modificar datos o eliminar elementos maestros de una tabla.
- La etiqueta `<tree>` funciona como la etiqueta raíz para definir listados o vistas de tipo lista.
- La etiqueta `<field>` se emplea para incorporar dinámicamente un elemento visual de datos o columna dentro de una sección del sistema (como una página de "Información General").
- La etiqueta `<group>` se utiliza específicamente para **organizar campos dentro de una "página" o sección del formulario**, asegurando que aparezcan ordenados y alineados bajo un título visual común.
- El elemento XML `<record>`, complementado con los atributos `model` e identificadores de campos, se encarga de insertar o actualizar registros estáticos y datos de ejemplo en las tablas de la base de datos al inicializar el componente.
- La jerarquía de accesos y la navegación de la barra superior del software se definen utilizando la etiqueta `<menuitem>`, vinculando la estructura del menú con su vista padre y con la acción de ventana correspondiente gobernada por el objeto del sistema `ir.actions.act_window` (el cual se encarga de inyectar la instrucción de abrir un formulario o listado específico).

Durante el proceso de diseño y adaptación fine-tuning de campos con `<field>`, si el desarrollador necesita inyectar un texto informativo gris aclaratorio cuando el campo se encuentra totalmente vacío, debe hacer uso del atributo `placeholder="..."`. Por otro lado, si requiere ****ocultar por completo la etiqueta de texto generada de forma automática**** por el framework para ganar espacio visual, debe activar el atributo XML `noLabel="1"`. Para salvaguardar la integridad de los datos financieros o sensibles, la recomendación técnica estándar para evitar que un perfil sin privilegios altere un dato crítico consiste en marcar el campo de forma explícita como ****no editable**** en el formulario XML según las reglas de negocio del rol de usuario.

4.2 Modificaciones Funcionales con Odoo Studio (Low-Code)

Para la realización de cambios funcionales ágiles, la herramienta integrada **Odoo Studio** ofrece la ventaja técnica de operar bajo un paradigma de desarrollo "low-code", posibilitando la adición de campos adicionales y la reestructuración de formularios **sin necesidad de poseer conocimientos de programación de bajo nivel**. La característica clave de Odoo Studio que garantiza de forma absoluta la estabilidad del sistema es que **crea de forma transparente vistas heredadas seguras**.

Para lograr este comportamiento, la plataforma se apoya en el modelo del sistema `ir.ui.view` (el cual almacena las definiciones en la base de datos) y genera de forma automatizada comandos y expresiones basados en **xpath** para inyectar los nuevos componentes en la posición exacta. Bajo ninguna circunstancia se debe realizar la modificación directa de las vistas estándar nativas del código base de Odoo, ya que esto provocaría la pérdida irreversible de las personalizaciones al aplicar futuras actualizaciones del fabricante; la directiva técnica estricta obliga a **utilizar siempre vistas heredadas**.

4.3 Informes Imprimibles y Lenguaje QWeb

El motor de informes integrado de Odoo utiliza el lenguaje de plantillas **QWeb**, un lenguaje avanzado desarrollado a partir de XML que tiene la capacidad de **combinar código HTML estándar, etiquetas dinámicas y variables de control de Python** para diseñar de forma flexible la interfaz de cualquier documento imprimible. El proceso físico de facturación u operaciones comerciales genera archivos de salida nativos en formato **PDF** al invocar la

acción "Imprimir factura". Técnicamente, esta operación se procesa en el servidor mediante plantillas QWeb vinculadas al motor del sistema a través de objetos de tipo `ir.actions.report`, asociando la numeración oficial y las plantillas corporativas del diario contable.

Adicionalmente, el módulo de ventas de Odoo dispone de un **diseñador integrado** que permite al usuario de negocio modificar en caliente el diseño visual de los presupuestos y facturas, habilitando la personalización directa del logotipo de la empresa, la tipografía de imprenta, la paleta de colores del documento y el diseño del bloque de detalles. Al estructurar estos reportes, la adición de una **cabecera (header) personalizada** cumple el fin corporativo de identificar con claridad meridiana el contenido y la procedencia legal de los datos del informe.

BLOQUE 5: GESTIÓN Y RELACIONES DE BASES DE DATOS (POSTGRESQL)

El motor de base de datos relacional oficial que soporta la persistencia de Odoo es **PostgreSQL**. Dentro del modelo de datos de Odoo, las relaciones lógicas entre las diferentes tablas (modelos) se parametrizan mediante tipos de campos relacionales específicos del ORM:

- **Many2one (Muchos a Uno):** Relación utilizada para vincular jerárquicamente un registro de un modelo con un único registro exclusivo de otra tabla externa (por ejemplo, asociar un Pedido de Venta a un Cliente determinado). En el esquema físico de PostgreSQL, estas relaciones se traducen en claves foráneas (Foreign Keys) que por convención de nomenclatura del framework **suelen terminar con el sufijo `_id`**.
- **Many2many (Muchos a Muchos):** Relación relacional que se debe emplear para vincular de forma bidireccional "N" elementos de un modelo con "N" elementos de otra tabla, generando de forma transparente una tabla intermedia de emparejamiento.

5.1 Extracción Analítica mediante Consultas SQL

Para la extracción directa y manipulación de datos almacenados en el motor relacional, se emplea el lenguaje estándar **SQL**. La sentencia mandatoria para extraer información específica y recuperar registros es la cláusula **SELECT**, cuya función técnica se centra en especificar detalladamente los ítems de datos exactos a recuperar, mientras que la cláusula complementaria **FROM** indica formalmente el nombre de la tabla física sobre la que se aplicará el filtrado y la extracción. Para modificar datos ya existentes en la base de datos se ejecuta la instrucción **UPDATE**.

Tablas Fundamentales del Maestro de Datos en Odoo:

- **res_partner**: Tabla núcleo e indispensable para centralizar todos los contactos (clientes, proveedores, empresas y empleados).
- **product_template**: Tabla maestra que almacena la plantilla conceptual y definición general de los productos.
- **product_supplierinfo**: Tabla relacional crítica para vincular los productos con sus respectivos proveedores y registrar los precios de compra pactados.

Cuando se programan consultas SQL complejas orientadas a la extracción y cruce de datos entre el inventario y el maestro de proveedores, el operador correcto para relacionar físicamente dos o más tablas en base a una columna coincidente es el comando **INNER JOIN**. Asimismo, si se procesan listados de información internacionalizados donde existan campos opcionales, el comando SQL que permite manejar de forma eficiente los valores nulos es la función **COALESCE**, la cual evalúa los argumentos en orden y selecciona automáticamente una traducción alternativa o valor por defecto si el campo principal se encuentra vacío. El filtrado de transacciones comerciales se opera aplicando **filtros rápidos** de tipo temporal o por fecha para aislar un periodo de tiempo concreto.

5.2 Procedimientos Almacenados en el Servidor

Los **Procedimientos Almacenados** son bloques de código estructurado escritos en lenguajes procedimentales propios del motor (específicamente **PL/pgSQL** en entornos PostgreSQL) que se guardan y se ejecutan directamente en el propio motor de la base de datos. Su gran ventaja técnica reside en que **el tiempo de respuesta se reduce de forma drástica debido a que el código se encuentra precompilado en el servidor**, permitiendo ejecutar múltiples instrucciones SQL complejas de forma conjunta y evitando el tráfico innecesario de red entre la aplicación y el almacenamiento.

En Odoo, se suelen explotar de forma selectiva para automatizar procesos repetitivos complejos en el servidor (por ejemplo, en un contexto VIP para calcular de forma automatizada un descuento complejo basado en el volumen de compras acumulado) o para garantizar la integridad referencial estricta y el rendimiento mediante restricciones (constraints) e índices a bajo nivel. No obstante, el framework de Odoo 18 **prioriza de forma absoluta el uso de módulos nativos en Python, ORM, campos de tipo compute y acciones programadas** para resolver la lógica de negocio del sistema habitual. El uso excesivo de procedimientos almacenados SQL directos conlleva el grave riesgo técnico de **aumentar críticamente el acoplamiento con la base de datos**, dificultando las migraciones de software y exigiendo la presencia de una documentación y baterías de pruebas rigurosas.

*(Directiva de Exclusión del Examen: Bajo ninguna circunstancia se deben utilizar sistemas de gestión de bases de datos ofimáticos de escritorio como ****Microsoft Access**** para dar soporte a un entorno ERP/CRM corporativo, dado que se descartan por completo al ser técnicamente inadecuados para soportar arquitecturas multiusuario, concurrencia de transacciones concurrentes ni escalabilidad empresarial).*

BLOQUE 6: CONTROL DE CALIDAD, PRUEBAS Y PLAN DE CONTINGENCIA

El objetivo primordial de ejecutar un plan de pruebas riguroso antes de poner el sistema ERP en el entorno de producción real es **asegurar de forma matemática que el sistema responde con exactitud a las necesidades funcionales de la empresa y garantizar que no aparezcan errores críticos** en caliente. En helpdesk o control de calidad, es una norma obligatoria que **cada prueba realizada durante la implantación sea documentada minuciosamente inyectando evidencias claras** como capturas de pantalla, ficheros planos de logs o métricas de rendimiento obtenidas, facilitando la auditoría técnica y la corrección de errores en caliente.

La tipología de pruebas de validación obligatorias se desglosa según el componente analizado:

- **Pruebas Funcionales:** Verifican específicamente que cada módulo individual cumple estrictamente con el comportamiento operativo previsto en el diseño (por ejemplo, verificar en un checklist elemental que al confirmar una orden de venta se genere automáticamente su factura correspondiente en el módulo financiero).
- **Pruebas de Integración:** Validan de forma exhaustiva la comunicación correcta y el flujo ininterrumpido de información entre los diferentes módulos del ERP (por ejemplo, comprobar formalmente que una operación de venta afecte de forma automática tanto al stock del almacén como a los asientos del módulo de contabilidad).
- **Pruebas de Usabilidad:** Tienen el fin de analizar y comprobar detalladamente si la interfaz gráfica de usuario resulta intuitiva, ergonómica y sencilla para los usuarios finales de la organización, cumpliendo el objetivo crítico de **reducir al mínimo los errores humanos** durante la introducción diaria de registros.
- **Pruebas de Rendimiento y Estrés:** Evalúan la velocidad de respuesta y la estabilidad del servidor midiendo el comportamiento del sistema ante la carga masiva de grandes volúmenes de datos simulados o el acceso masivo de múltiples usuarios concurrentes conectados de forma simultánea.
- **Pruebas de Seguridad:** Acción de validación orientada a revisar y confirmar que los permisos de acceso y grupos de usuarios restringen el acceso a módulos sensibles (por ejemplo, validar de forma estricta que un perfil de usuario con restricciones en el módulo de ventas tenga totalmente bloqueado el acceso a las funciones del módulo de contabilidad).
- **Pruebas de Compatibilidad:** Adquieren una relevancia crítica en escenarios modernos de teletrabajo y movilidad laboral, y se encargan de verificar el correcto funcionamiento del software a través de diversos navegadores web (Chrome, Firefox, Edge), sistemas operativos de servidor y terminales de dispositivos móviles.

Si durante el proceso de pruebas o tras una actualización menor se identifica una desviación crítica entre los objetivos planteados y el comportamiento real del ERP, el equipo técnico debe registrar de forma inmediata la incidencia documentando el fallo. Si los pequeños cambios o parches en caliente no consiguen solucionar la anomalía, la metodología oficial dictamina ejecutar una estrategia de contingencia basada en **realizar una "vuelta atrás" (rollback) del código del módulo** para restaurar el entorno al estado operativo previo que funcionaba correctamente.

BLOQUE 7: DEPURACIÓN (DEBUGGING) Y DIAGNÓSTICO DE FALLOS

Para realizar el diagnóstico técnico y la modificación avanzada de tablas y campos directamente sobre la base de datos de Odoo a través de formularios, el desarrollador debe activar previamente el **icono de debug o modo desarrollador**. Este modo técnico se habilita de forma directa accediendo a la ruta **Ajustes → Activar modo desarrollador**, o bien manipulando la barra de direcciones del navegador web inyectando el parámetro **? debug=1** dentro de la URL de acceso de la sesión de trabajo. En el lado del cliente (navegador web), la depuración visual y la inspección en caliente de la estructura de la página se facilita mediante el uso de las

herramientas de desarrollo del propio navegador (F12) o instalando la extensión especializada denominada **Odoo Debug Manager**.

En el entorno de desarrollo del lado del servidor, se exponen dos técnicas de diagnóstico contrapuestas:

- **Función `print()`:** Técnica tradicional que permite visualizar de forma rápida los valores de las variables imprimiendo texto plano directamente por la consola del servidor. Permite validar de forma sencilla por qué rama condicional de una estructura `if-else` transcurre la ejecución, pero presenta el gran inconveniente técnico de ralentizar la velocidad de ejecución del código fuente.
- **Depurador Interactivo (`pdb`):** Herramienta avanzada cuya principal ventaja técnica radica en permitir la ejecución interactiva y paso a paso del programa, posibilitando la inspección de variables en tiempo real y asegurando una corrección de fallos mucho más rápida y profesional.

Al enfrentarse a una ventana de detalles de error generada por el servidor Odoo, el técnico debe seguir un protocolo ordenado para aislar la causa raíz: el primer paso recomendado consiste en **abrir el contexto específico que indica el mensaje de error** (generalmente localizando un fichero de código y una línea exacta). La traza técnica detallada del error (stack trace) proporciona la información vital del itinerario del fallo junto con el mensaje explicativo contextualizado de la excepción.

BLOQUE 8: SEGURIDAD, AUDITORÍA Y GESTIÓN DE LOGS

La seguridad y el control estricto de accesos a nivel de datos se articula en Odoo mediante la definición del modelo relacional de seguridad **ir.model.access**, el cual se declara de forma normalizada mediante la carga de un archivo estructurado en formato ****CSV****. En este componente se definen con precisión quirúrgica los permisos de lectura, escritura, creación y desconexión (enlaces relacionales) por cada grupo o rol de usuario de la corporación. Un pilar básico de configuración de la seguridad de la infraestructura consiste en aislar los accesos declarando de forma unívoca el ****Usuario de acceso a la BD del sistema de gestión empresarial****, impidiendo el uso de cuentas genéricas de administración del motor.

Para la auditoría y diagnóstico de eventos del sistema, un **log** es un fichero de texto plano almacenado en el servidor que registra de forma inmutable las acciones, errores y eventos de comportamiento de la aplicación informática, asociando siempre campos obligatorios como la fecha y hora exacta del suceso, el proceso que lo invocó y la categoría o nivel de criticidad del evento. Los niveles de severidad de los logs se tipifican bajo estándares internacionales: el nivel **INFO** indica funcionamiento normal y trazas de transacciones cotidianas; el nivel **WARN** alerta sobre posibles anomalías que no bloquean la ejecución; y el nivel **ERROR (ERR)** identifica fallos explícitos y excepciones críticas de la aplicación.

El control exhaustivo de los logs del servidor Odoo se realiza editando el archivo de configuración global del entorno denominado **odoo.conf**. A nivel de base de datos PostgreSQL en sistemas de producción Linux, el fichero de log se ubica por defecto en la ruta física del sistema de archivos **.../data/log/**. El comportamiento y el formato de las trazas del motor de la base de datos se gobiernan mediante parámetros de configuración específicos declarados dentro del fichero central de parámetros **postgresql.conf**:

- **log_line_prefix**: Parámetro técnico encargado de definir exactamente qué información descriptiva de control debe aparecer fijada al inicio de cada línea del archivo de log (por ejemplo, el identificador de proceso PID, el usuario del sistema conectado o la base de datos exacta atacada).
- **log_connections** y **log_disconnections**: Parámetros booleanos que fuerzan al motor a registrar de forma inmutable la identidad de cada usuario que entra y sale del servidor de base de datos.
- **log_min_duration_statement**: Parámetro avanzado de rendimiento que registra de forma automática en los logs todas aquellas consultas de selección o modificación SQL que superen un tiempo de ejecución límite determinado, aislando cuellos de botella.

BLOQUE 9: EXPLOTACIÓN DE DATOS E INTELIGENCIA DE NEGOCIO (BI)

La extracción de información en un entorno ERP-CRM consiste en preparar y presentar de forma estructurada la información almacenada en el repositorio central de datos. Odoo cuenta con mecanismos internos que facilitan la extracción y presentación directa de información por pantalla mediante el uso de vistas interactivas, listados planos o gráficos dinámicos que garantizan la coherencia de la información al beber de una base de datos centralizada. Muchas de estas vistas permiten una ordenación rápida de los registros haciendo clic directo sobre el encabezado de las columnas de la tabla.

Para la explotación avanzada de la información, el usuario de Odoo dispone de la opción de exportación a formatos externos (Excel, CSV o Word) seleccionando las líneas deseadas del listado y ejecutando la opción "Exportar" dentro del menú Acción (pudiendo elegir campos del maestro como la referencia del pedido). No obstante, se debe considerar que los datos exportados a un fichero Excel externo **quedan completamente desconectados del ERP-CRM**, lo que significa que no se actualizarán de forma automática ante cambios en el sistema salvo que se proceda a realizar una nueva exportación física de los datos.

9.1 Inteligencia de Negocio Interna: Odoo Spreadsheet y Vistas Pivot

Para evitar la desconexión de los datos y facilitar la toma de decisiones basada en datos objetivos, las versiones modernas de Odoo integran la herramienta **Odoo Spreadsheet**. Esta funcionalidad consiste en una hoja de cálculo completamente integrada en el propio ERP que permite confeccionar KPIs analíticos y paneles dinámicos en tiempo real.

Los paneles se nutren de forma directa mediante la inserción de las denominadas **Vistas Pivot (Tabla Pivotante)**. Esta visualización avanzada destaca en el módulo de ventas debido a que permite agregar de forma flexible nuevos cálculos, métricas y dimensiones dinámicas a partir de un listado plano de información. Asimismo, la capa de presentación visual permite desplegar **Vistas Gráficas (<graph>)** para generar representaciones gráficas de datos y cuadros de mando interactivos cuyo tipo de medición se puede alterar dinámicamente utilizando el menú interactivo de "Medidas" de la barra de herramientas.

9.2 Herramientas Externas de Reporting y Business Intelligence (BI)

La finalidad principal de la Inteligencia de Negocio (BI) radica en **transformar datos en bruto en información útil y estructurada para la toma de decisiones estratégicas**. Frente a los informes básicos y rígidos que proporciona de forma nativa un ERP tradicional, las herramientas externas de BI ofrecen la ventaja competitiva de permitir realizar análisis de datos multidimensionales y cuadros de mando interactivos infinitamente más potentes y en tiempo real.

Cuando la organización opta por un modelo de extracción mediante un generador de informes externo (cuyo objetivo técnico es generar y servir informes bajo una arquitectura cliente-servidor robusta, al estilo de Crystal Reports), el paso previo e indispensable antes de acometer el diseño del informe personalizado consiste en ****conocer al detalle la base de datos y mapear la localización exacta de la información**** que se desea exportar.

La integración con herramientas analíticas externas obliga a seguir directrices de seguridad de la infraestructura de datos: se recomienda de forma mandatoria crear en el motor PostgreSQL un ****Rol de solo lectura (RO)**** exclusivo para las herramientas de BI, asegurando que los analistas externos ataquen la base de datos de forma segura sin privilegios de alteración o borrado de datos de producción. Las alternativas del mercado se dividen según su arquitectura:

- **JasperSoft Studio / Jasper Reports:** Herramienta externa de código abierto que dispone de una versión Community ideal para confeccionar informes altamente corporativos y personalizados directamente sobre la base de datos. Proporciona recursos avanzados para introducir elementos visuales, indicadores de rendimiento complejos y agrupaciones jerárquicas legibles (por ejemplo, agrupar productos según su proveedor de origen). La sección denominada ****Detail (Detalle)**** de la plantilla del informe es el elemento encargado de albergar y renderizar los datos y registros provenientes de la base de datos. Se conecta mediante JDBC empleando por defecto el puerto de red ****5432**** de PostgreSQL.
- **Pentaho:** Potente plataforma alternativa de Business Intelligence libre que dispone de una versión community muy extendida para procesos analíticos.
- **Power BI:** Herramienta analítica de la suite de Microsoft que permite diseñar paneles interactivos avanzados conectándose directamente sobre el motor PostgreSQL de Odoo mediante el uso de proveedores de datos como **Npgsql**.

A nivel de modelado de datos para BI masivo y almacenamiento analítico, el sistema puede evolucionar hacia un entorno de **Data Warehouse**, encargado de integrar los flujos de datos provenientes del ERP, CRM y e-commerce de la compañía en un repositorio central masivo unificado. Si la arquitectura se enfoca hacia un departamento o área de negocio reducida mediante un proceso de extracción, transformación y carga (ETL), el modelo resultante se denomina técnicamente **Data Mart**.

Por último, para el control técnico y de sistemas en tiempo real, el equipo de sistemas puede desplegar un enfoque de **Monitorización de las Infraestructuras** para analizar el espacio en disco, la conectividad de la red y el funcionamiento del hardware del servidor mediante soluciones de código abierto como **Nagios**. Cuando esta monitorización de infraestructura y logs se integra con la herramienta **Prometheus**, la suite de software de referencia elegida para crear de forma unificada los cuadros de mando y dashboards interactivos en tiempo real es **Grafana**.

BLOQUE 10: GESTIÓN DE INCIDENCIAS EN LA IMPLANTACIÓN (HELPDESK)

Durante la fase de despliegue y puesta en marcha del sistema de gestión empresarial, es un requisito imprescindible contar con un canal técnico estructurado o **sistema de helpdesk** para que los técnicos informáticos puedan recoger, evaluar, clasificar y documentar de manera formal cada incidencia técnica reportada por los usuarios de la organización.

Dentro de la operativa diaria de soporte, el flujo metodológico formalizado dicta que, inmediatamente tras completar el paso de **Clasificar la incidencia** según su impacto y naturaleza, el técnico debe proceder de forma directa a ****Realizar la solución técnica que ya se encuentre previamente documentada**** en el repositorio de conocimiento del equipo, agilizando los tiempos de resolución y garantizando la homogeneidad del soporte.

Las anomalías reportadas en el entorno se catalogan según su naturaleza y origen técnico: los problemas físicos o de comunicaciones se asocian a la infraestructura, mientras que aquellas que presentan una mayor complejidad de análisis e inspección son las denominadas **Incidencias de Aplicación (o de Negocio)**. Esta alta complejidad de resolución se debe expresamente a que sus causas no se manifiestan mediante un error explícito de código del programa, sino que suelen estar provocadas por una **mala parametrización funcional del sistema o por una errónea implantación inicial de las reglas de negocio de la empresa** en los flujos del software, requiriendo una auditoría del proceso de trabajo frente a los objetivos corporativos planteados. La documentación exhaustiva e inmutable de cada incidencia es mandatoria, ya que permite cimentar una robusta base de información técnica y de conocimiento operativo que pueda ser utilizada de forma eficiente ante fallos futuros en el sistema.